

FunctorFlow User’s Manual

Language Reference and Implementation Guide for v0

Sridhar Mahadevan

March 28, 2026

Contents

1	Purpose of this manual	3
2	What FunctorFlow is	3
2.1	Current scope of v0	3
2.2	What FunctorFlow does not yet claim	4
3	Language model	4
4	Surface forms	4
5	Core constructs	5
5.1	Objects	5
5.2	Morphisms	5
5.3	Compositions	6
5.4	Kan extensions	6
5.5	Obstruction losses	6
6	The Diagram abstraction	7
6.1	Role of diagrams	7
6.2	Diagram construction API	7
7	Ports, port typing, and adapters	7
7.1	Ports	7
7.2	Port typing	8
7.3	Adapters	8
8	Subdiagram inclusion and namespaces	8
9	Macros and libraries	8
9.1	Macro layer	8
9.2	Typed configuration objects	9
9.3	Adapter libraries	9
9.4	Tutorial libraries	9
9.5	Structured language-model support	9

10	Compilation and execution	10
10.1	Normalized IR	10
10.2	Backends	10
10.3	Language guarantee in v0	10
11	Notebook and demo layer	11
12	Proof interface	11
13	Worked examples	11
13.1	Minimal KET block	11
13.2	Composite planning workflow	12
13.3	Structured language-model duality	12
13.4	Structured language-model experiment	13
13.5	Notebook-level corpus comparison	13
13.6	Mini-Sudoku as a structural consistency problem	13
14	Current package structure	14
15	Recommended usage style	15
16	Versioning note	15
17	Closing remark	15

1 Purpose of this manual

FunctorFlow is a small categorical language for expressing deep learning architectures as typed diagrams, then compiling those diagrams into executable artifacts. The slide deck gives the motivation, but it does not fully define the language. This manual fills that gap.

The goals of this document are:

- to define the current FunctorFlow v0 language constructs carefully,
- to explain how those constructs map onto the Python implementation,
- to document the current package-level APIs and usage patterns,
- to provide stable terminology as the language evolves.

This is a *living manual*. It should be updated as new language features are added.

2 What FunctorFlow is

FunctorFlow is centered on the compiler picture

Diagram / Spec $\longrightarrow$ Categorical IR $\longrightarrow$ Executable Backend.

In practical terms, a user writes a *diagram* consisting of typed objects, morphisms, compositions, Kan extensions, and structural losses. FunctorFlow then normalizes that diagram into an intermediate representation and lowers it to one of the currently supported backends.

2.1 Current scope of v0

FunctorFlow v0 intentionally aims for a narrow but useful slice of the larger vision. The current implementation supports:

- typed objects and morphisms,
- named diagram construction,
- left and right Kan extensions as first-class operations,
- obstruction losses for non-commuting paths,
- semantic ports with port typing,
- explicit adapters and adapter libraries,
- namespaced inclusion of subdiagrams,
- macro libraries and tutorial libraries,
- lowering to plain Python execution and optional Torch modules,
- reusable completion and structured-language-model duality macros,
- a unified structured language-model experiment path for block prediction and denoising completion,
- notebook generation and a proof-of-concept Lean certificate path.

2.2 What FunctorFlow does not yet claim

FunctorFlow v0 does *not* yet claim:

- full formal semantics for arbitrary small categories,
- full theorem-proving coverage of all deep learning behavior,
- automatic training pipelines for every domain in the larger repo,
- a complete parser separate from the Python API.

The current language is therefore best understood as a carefully designed *embedded DSL* in Python with a categorical intermediate representation.

3 Language model

FunctorFlow has three levels:

1. **Core language.** Objects, morphisms, compositions, Kan extensions, obstruction losses, ports, and adapters.
2. **Vocabulary layer.** Named macro builders such as KET, DB, GT, BASKET, ROCKET, and Democritus blocks.
3. **Execution layer.** Compilation to a normalized IR and then to callable Python or Torch.

The primary user-facing artifact is a `Diagram`.

4 Surface forms

FunctorFlow is currently expressed through Python constructors and methods rather than a separate parser. The effective surface forms are:

```
from FunctorFlow import Diagram, Object, Morphism

D = Diagram("MyDiagram")

x = D.object("Tokens", kind="sequence")
y = D.object("Latent", kind="representation")

f = D.morphism("encode", x, y, implementation=my_encoder)
g = D.compose("encode", name="encode_once_more")

k = D.left_kan(
    name="aggregate",
    source="Messages",
    along="Incidence",
    target="Contextualized",
    reducer="mean",
)

r = D.right_kan(
    name="repair",
```

```

    source="PartialPlan",
    along="ObservationMap",
    target="CompletedPlan",
    reducer="mean",
)

loss = D.obstruction_loss(
    name="db_loss",
    paths=[("path_a", "path_b")],
)

```

Two styles are idiomatic:

- **Detached declaration style:** construct `Object` and `Morphism` values separately, then add them to a diagram.
- **Diagram-first style:** create everything directly through a `Diagram` instance.

5 Core constructs

5.1 Objects

An `Object` names a typed semantic interface. In code, an object has:

- a `name`,
- a `kind`,
- an optional `shape`,
- a human-readable `description`,
- arbitrary `metadata`.

Objects are not necessarily tensors. They represent semantic slots such as:

- token states,
- neighborhood indices,
- plan fragments,
- observation contexts,
- latent manifolds,
- repaired plans.

5.2 Morphisms

A `Morphism` is a typed arrow from one object to another. It may carry a direct implementation or an implementation key resolved later by the compiler.

Semantically, morphisms represent learned or fixed transformations, including:

- encoder layers,

- projection maps,
- neighborhood lifts,
- workflow transitions,
- repair operators.

5.3 Compositions

A **Composition** names a composed path through previously declared morphisms. The compiler validates that the chain endpoints align. This gives the language a first-class way to talk about paths, not only primitive arrows.

5.4 Kan extensions

FunctorFlow treats Kan extensions as first-class operations.

Left Kan extension. `left_kan(source, along, target)` expresses structured aggregation. This is the native FunctorFlow home for:

- attention,
- pooling,
- geometric message passing,
- context integration,
- fragment aggregation.

Right Kan extension. `right_kan(source, along, target)` expresses completion or repair under structure. This is the language slot for:

- denoising,
- masked completion,
- candidate-plan repair,
- partial-view reconciliation.

In v0, both left and right Kan extensions are represented as named operations with a **direction**, a **source**, an **along** map, an optional **target**, a **reducer**, and metadata.

5.5 Obstruction losses

ObstructionLoss turns diagrammatic inconsistency into an explicit loss. It compares named paths that ought to agree. This gives a direct home to DB-like commutativity constraints.

The important language point is that inconsistency is not treated as an afterthought; it is a first-class architectural object.

6 The Diagram abstraction

6.1 Role of diagrams

A **Diagram** is the unit of specification, reuse, inclusion, compilation, and documentation. A diagram is the thing users should think in.

Every diagram owns:

- an ordered dictionary of objects,
- an ordered dictionary of operations,
- an ordered dictionary of losses,
- an ordered dictionary of ports,
- implementation registries for morphisms, reducers, comparators, and adapters.

6.2 Diagram construction API

The core construction methods are summarized in Table 1.

Table 1: Core FunctorFlow language operations

Method	Meaning
<code>object(...)</code>	Declare an object owned by the diagram.
<code>morphism(...)</code>	Declare a typed arrow, optionally binding an implementation.
<code>compose(...)</code>	Name a composed path through existing operations.
<code>left_kan(...)</code>	Add a left Kan aggregation operation.
<code>right_kan(...)</code>	Add a right Kan completion operation.
<code>obstruction_loss(...)</code>	Add a structural loss over named paths.
<code>expose_port(...)</code>	Publish a semantic interface for later wiring.
<code>register_adapter(...)</code>	Declare a legal type bridge between ports.
<code>include(...)</code>	Include another diagram under a namespace.
<code>to_ir()</code>	Normalize the diagram into categorical IR.

7 Ports, port typing, and adapters

7.1 Ports

A **Port** is a named semantic interface exposed by a diagram. A port has:

- a name,
- a reference into the diagram,
- a kind,
- a `port_type`,
- a direction,

- optional metadata.

Ports are how composite systems should wire diagrams together. Users should prefer port names such as `input`, `relation`, `output`, or `loss` over reaching into internal operation names.

7.2 Port typing

Port typing is the current FunctorFlow mechanism for guarding composition. A connection is valid if:

- the source and target port types match exactly, or
- a declared adapter bridges the types.

This is intentionally strict. FunctorFlow prefers explicit semantic contracts over permissive string-based wiring.

7.3 Adapters

An **Adapter** specifies a legal coercion from one port type to another. Adapters are explicit program artifacts. They can be registered directly on a diagram or installed through reusable adapter libraries.

This allows the language to stay typed while still supporting intentional bridges such as:

- contextualized messages $\rightarrow$ plan candidates,
- message neighborhoods $\rightarrow$ contextual states,
- candidate plans $\rightarrow$ repaired plans.

8 Subdiagram inclusion and namespaces

FunctorFlow supports namespaced inclusion through `Diagram.include(...)`. When a child diagram is included, its objects, operations, losses, and ports are copied into the parent namespace with stable renamed references.

This mechanism is important for two reasons:

- it lets large systems be built from smaller named blocks,
- it provides the structural basis for BASKET/ROCKET-style workflows.

The result of inclusion is an `IncludedDiagram` handle, which provides access to renamed objects, operations, losses, and ports.

9 Macros and libraries

9.1 Macro layer

The macro layer provides reusable named blocks on top of the core language. Current built-in macro builders include:

- `ket_block`,

- `completion_block`,
- `structured_lm_duality`,
- `db_square`,
- `gt_neighborhood_block`,
- `basket_workflow_block`,
- `rocket_repair_block`,
- `democritus_gluing_block`,
- `basket_rocket_pipeline`.

Macros are not a second semantics. They are a vocabulary layer over the same core diagram language.

9.2 Typed configuration objects

Macros are parameterized by typed configuration dataclasses. This matters because FunctorFlow is a language, and language constructs should be explicit. Config types make the available parameters visible and stable.

9.3 Adapter libraries

Adapter libraries package reusable coercion vocabularies. A diagram can install an adapter library with one call rather than registering each bridge manually.

9.4 Tutorial libraries

Tutorial libraries package:

- a curated subset of macros,
- the adapter libraries those macros expect,
- a higher-level story for a tutorial audience.

This provides a useful intermediate layer between the raw DSL and application-specific code.

9.5 Structured language-model support

FunctorFlow now has two closely related language-modeling layers.

First, `ket_lm.py` provides the basic KET language-model path:

- corpus loaders,
- KET attention-as-left-Kan heads,
- small executable language-model stacks.

Second, `structured_lm.py` packages the block-versus-denoise view explicitly:

- `task="block"` corresponds to direct future-block prediction,
- `task="denoise"` corresponds to structured completion of a corrupted future block,
- the experiment payload reports metrics such as block perplexity, first-offset perplexity, and denoising reconstruction accuracy.

This is the current closest FunctorFlow realization of the structured language modeling story used throughout the surrounding research program.

10 Compilation and execution

10.1 Normalized IR

The current normalized IR records:

- objects,
- operations,
- losses,
- ports.

Operations in the current IR are represented as one of:

- morphism,
- composition,
- Kan extension.

10.2 Backends

FunctorFlow v0 currently supports two execution backends:

- `compile_to_callable`, a backend-neutral Python executor,
- `compile_to_torch`, a Torch lowering path when `torch` is installed.

The backend-neutral executor is important because it lets the semantic layer be tested independently of full Torch availability.

10.3 Language guarantee in v0

The language-level guarantee in v0 is modest but meaningful:

If a FunctorFlow diagram is well-formed, the compiler should be able to normalize it into an IR whose named references, ports, and lowering structure are explicit and inspectable.

This is a software-architecture guarantee, not a performance theorem.

11 Notebook and demo layer

FunctorFlow can generate notebooks for:

- tutorial libraries,
- KET block demonstrations,
- PTB and Wiki-2 KET language-model runs,
- PTB and Wiki-2 model comparisons,
- the KET block-duality notebook, which now also drives real Transformer / GT / KET comparisons on PTB, Wiki-2, or Wiki-103 and renders comparison plots,
- the predict-detach regime demonstration,
- a mini-Sudoku structural-constraint demo.

The notebook layer matters because it provides a tutorial-facing presentation of the language while keeping the real implementation in the package modules.

In practice, the generated notebooks are now useful in two distinct modes:

- **smoke mode**, where short runs validate the language surface and local execution path,
- **comparison mode**, where the same notebook is retargeted to a real corpus and used to produce executable plots and model-family comparisons.

12 Proof interface

FunctorFlow now includes a proof-of-concept Lean layer under `FunctorFlow/proofs/`. The current proof interface is intentionally narrow. It does not prove anything about learning performance. Instead, it checks part of the compilation story itself.

The current proof path:

1. serialize a diagram into a small Lean IR,
2. state basic well-formedness conditions on operations and ports,
3. state soundness conditions on the lowering artifact,
4. emit generated Lean certificates that typecheck in the local Lean project.

This is best understood as a *proof-carrying compiler prototype*.

13 Worked examples

13.1 Minimal KET block

```

from FunctorFlow import KETBlockConfig, ket_block, compile_to_callable

diagram = ket_block(
    KETBlockConfig(
        name="TutorialKET",
        source_object="EdgeValues",
        relation_object="TokenIncidence",
        target_object="TokenStates",
    )
)

executor = compile_to_callable(diagram)

```

This example illustrates the intended pattern:

- use a named macro for a canonical construction,
- inspect or expose ports at the diagram level,
- compile the result using a backend.

13.2 Composite planning workflow

```

from FunctorFlow import BasketRocketPipelineConfig, basket_rocket_pipeline

workflow = basket_rocket_pipeline(
    BasketRocketPipelineConfig(name="PlanAndRepair")
)

```

This example demonstrates that FunctorFlow is not only for language modeling. It is also designed for composite workflow diagrams of the BASKET/ROCKET kind.

13.3 Structured language-model duality

```

from FunctorFlow import StructuredLMDualityConfig, structured_lm_duality

diagram = structured_lm_duality(
    StructuredLMDualityConfig(name="StructuredLM")
)

print(diagram.port("context")) # left-Kan aggregation output
print(diagram.port("completed")) # right-Kan completion output

```

This macro makes one of the guiding FunctorFlow symmetries explicit:

- left Kan handles attention-style context aggregation,
- right Kan handles denoising-style completion under constraints,
- the paired diagram exposes both surfaces as stable ports.

13.4 Structured language-model experiment

```
from FunctorFlow import (
    KETStructuredLanguageModelConfig,
    run_structured_language_model_experiment,
)
from FunctorFlow.ket_lm import pick_device

config = KETStructuredLanguageModelConfig.historical_smoke(
    "ptb",
    task="denoise",
)
result = run_structured_language_model_experiment(
    config,
    device=pick_device("cuda"),
)

result["task"], result["eval"]["block_ppl"]
```

This example shows the current recommended path when the goal is not merely to inspect a diagram, but to run the paired block/denoise KET story through a real Torch-backed training loop.

13.5 Notebook-level corpus comparison

The generated `ket_block_duality_demo.ipynb` notebook now extends the basic left-Kan/right-Kan architectural demo with a real corpus-level model comparison. In its current form the notebook can:

- run the small duality smoke experiment,
- switch to `compare_language_models(...)` for PTB, Wiki-2, or Wiki-103,
- render a validation-perplexity bar chart and training-loss curves at the end.

This matters pedagogically because the same notebook now links:

- the categorical interpretation,
- the executable KET block-vs-denoise core,
- and a concrete Transformer / GT / KET benchmark comparison.

13.6 Democritus causal-document pipeline

FunctorFlow now includes a standalone Democritus implementation for running a document through a typed causal-discovery workflow. The current public path supports both PDF-backed and plain-text inputs and produces a structured artifact bundle rather than only an in-memory manifold.

```
from pathlib import Path

from FunctorFlow.democritus import (
    DemocritusPipelineConfig,
    run_democritus_pipeline_from_pdf,
)
```

```

artifacts = run_democritus_pipeline_from_pdf(
    Path("data/democritus/sample.pdf"),
    config=DemocritusPipelineConfig.smoke(domain_name="manual_demo"),
    outdir=Path("FunctorFlow/democritus_runs/manual_demo"),
)

print(len(artifacts.relational_triples))
print(artifacts.output_files["manifold"])

```

The Democritus pipeline currently performs the following stages:

- root-topic discovery over the document text,
- topic-graph expansion,
- causal-question generation for topic paths,
- causal-statement generation,
- relational-triple extraction,
- global causal-manifold construction,
- artifact export, including visual summaries.

The exported artifact bundle now includes:

- `root_topics.txt`,
- `topic_graph.jsonl`,
- `causal_questions.jsonl`,
- `causal_statements.jsonl`,
- `relational_triples.jsonl`,
- `manifold.npz` and `manifold_summary.json`,
- `causal_manifold.png`,
- a `local_causal_models/` directory containing a few sample local causal-model PNGs,
- `local_causal_models.json`, which records the selected focus entities and the rendered PNG paths,
- `democritus_diagram.json`, which serializes the FunctorFlow diagram itself.

Conceptually, this example is useful because it shows how the FunctorFlow language can be used for more than neural-network blocks alone. In Democritus,

- GT-style neighborhood refinement appears at the local-claim level,
- Democritus gluing appears as a named sheaf-like manifold assembly step,
- and the output surface mixes symbolic artifacts, numeric embeddings, and rendered causal figures.

13.7 Mini-Sudoku as a structural consistency problem

Sudoku is a particularly good FunctorFlow example because it is fundamentally a *constraint-structured* task. For a 4×4 Sudoku with 2×2 blocks, each cell participates in three overlapping contexts:

- a row,
- a column,
- a block.

The structural rule is simple:

No digit should be repeated within any row, column, or block.

In FunctorFlow language, this becomes:

- an object for cell digits,
- three relation objects for row, column, and block incidence,
- left-Kan aggregations into row-, column-, and block-level summaries,
- explicit morphisms that turn those summaries into violation counts.

```
from FunctorFlow.sudoku_demo import (
    analyze_sudoku_board,
    build_sudoku_constraint_diagram,
)

diagram = build_sudoku_constraint_diagram()
print(diagram.summary())

report = analyze_sudoku_board([
    0, 1, 2, 3,
    2, 3, 0, 1,
    1, 0, 3, 2,
    3, 2, 1, 0,
])

report["is_consistent"], report["total_duplicates"]
```

This example illustrates several important points about FunctorFlow v0:

- the language is useful even when the main story is not language modeling,
- left Kan extensions naturally express aggregation over structured incidence relations,
- structural consistency can be compiled into explicit executable artifacts,
- FunctorFlow is strongest, in v0, when the task itself is organized by visible compositional constraints.

The package also includes a smoke-scale training demo around this structure. There, a plain Transformer and a GT-style reasoner are compared on generated 4×4 Sudoku puzzles while using the same board structure and triangle-based consistency signal.

14 Current package structure

Table 2 summarizes the current top-level package structure.

Table 2: Top-level FunctorFlow package structure

Path	Role
<code>core.py</code>	Core language constructs and diagram assembly.
<code>compiler.py</code>	IR execution and Torch lowering.
<code>macros.py</code>	Named block vocabulary.
<code>adapter_library.py</code>	Reusable adapter packaging.
<code>tutorial_library.py</code>	Curated tutorial-level libraries.
<code>democritus.py</code>	Typed Democritus document pipeline, artifact writer, and causal-manifold exporter.
<code>ket_lm.py</code>	KET language-model implementations and presets.
<code>structured_lm.py</code>	Unified block-vs-denoise structured LM experiments.
<code>lm_compare.py</code>	Transformer / GT / KET comparison harnesses.
<code>predict_detach_demo.py</code>	Predict-detach regime demo.
<code>sudoku_demo.py</code>	Mini-Sudoku structural-constraint demo.
<code>notebook_renderer.py</code>	Notebook generation.
<code>proof_interface.py</code>	Lean certificate export.
<code>proofs/</code>	Lean proof-of-concept project.
<code>examples/</code>	Runnable package examples, including <code>democritus_demo.py</code> for PDF/text causal-document runs.
<code>tests/</code>	Regression and smoke tests.
<code>data/</code>	Package-local PTB and Wiki-2 data, with the same loaders also supporting Wiki-103 when present locally or in the repo root.

15 Recommended usage style

For users writing new FunctorFlow code, the recommended progression is:

1. start with macros when a named system concept already exists,
2. use ports rather than internal operation names for composition,
3. add adapters explicitly when semantic type coercions are intended,
4. include subdiagrams under namespaces rather than flattening by hand,
5. drop to the raw `Diagram` API only when the macro layer is not expressive enough.

16 Versioning note

This manual documents FunctorFlow v0 as it exists in the current repository. The implementation is evolving quickly. Whenever a new feature materially changes the language surface, this manual should be updated alongside the code.

17 Closing remark

FunctorFlow is meant to be more than a collection of helper classes. It is a language for expressing a family of related deep learning constructions in one compositional vocabulary. The purpose of this manual is to make that language explicit.